SQL Explanation Grader: baseline vs RAG

Experiment **rag_vs_baseline_dev** · split **dev** (50 items) · generator **llama3.2:3b** · judge **qwen2.5:3b** (rubric) · generated 2026-10-26 18:08

How to read this report. "Judge approval" is the share of explanations an LLM judge graded good. It is an automated, unverified measurement, not ground truth. Deterministic checks are rule-based heuristics computed from the SQL structure. Agreement of the judge with reference labels is reported separately (results/rag/judge_agreement.json).

Headline comparison

Metric	Baseline (prompt only)	RAG
Judge approval (95% CI)	100.0% (50/50) [93-100]	98.0% (49/50) [90-100]
Approval, abstentions count as fail	100.0% (50/50) [93-100]	98.0% (49/50) [90-100]
Rubric pass (judge's own scores)	100.0% (50/50) [93-100]	98.0% (49/50) [90-100]
Mean correctness (1-5)	4.98	4.92
Mean completeness (1-5)	4.98	4.92
Mean clarity (1-5)	4.98	4.92
Mean groundedness (1-5)	4.98	4.92
Explanations with unsupported claims (judge)	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Deterministic check coverage	0.96	0.98
Invented numbers (rule-based)	10.0% (5/50) [4-21]	12.0% (6/50) [6-24]
Raw SQL keywords in text	10.0% (5/50) [4-21]	66.0% (33/50) [52-78]
Abstentions	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Generation errors	0	0
Mean end-to-end latency (s)	6.6	17.5
Mean retrieval time (s)	0.00	1.49
Mean input tokens	571	2647
Mean output tokens	104	266

AI-assisted reference labels (not human): share labelled good

	Baseline	RAG
Labelled good	20/50	10/50
Paired: only baseline good / only RAG good	16	6
RAG minus baseline (95% bootstrap CI)	-20.0 pts (-38.0 to -2.0)	McNemar p = 0.052

Labels and their decision rules: data/labels/. They are not produced by the judge in this report.

Paired comparison (same items, both judged)

50 pairs. Both good: 49; both bad: 0; only baseline good: 1; only RAG good: 0. Approval difference RAG minus baseline: -2.0 points (95% bootstrap CI -6.0 to +0.0). Exact McNemar p = 1.0.

Dimension	Mean RAG minus baseline	95% CI
correctness	-0.06	-0.18 to +0.04
completeness	-0.06	-0.18 to +0.04
clarity	-0.06	-0.18 to +0.04
groundedness	-0.06	-0.18 to +0.04

Grounding and retrieval (RAG condition)

Metric	Value
Answers with at least one citation	90.0% (45/50) [79-96]
Citation validity (cited source was provided)	1.000
Answers citing a source that was not provided	0.0% (0/50) [0-7]
Share of sentences carrying a citation	0.447
Lexical support of cited sentences (heuristic)	0.806
Model said evidence was insufficient	0
Retrieval feature coverage	1.000
Evidence judged sufficient before generation	100.0% (50/50) [93-100]
Grounding status counts	{"grounded": 45, "weakly_grounded": 5}

Retrieval evaluation (no LLM involved)

Project-authored by Claude (AI-assisted) on 2026-10-26 by reading the section headings of each corpus document. Not human-reviewed yet. Treat retrieval scores computed with it as rule-based estimates, not human relevance judgements.

Chunk	Strategy	Mode	MRR	Recall@5	Prec@5	nDCG@5	Ctx recall	ms/query
400	decomposed	dense	0.824	0.925	0.556	0.616	0.892	161
400	decomposed	bm25	0.642	0.836	0.568	0.560	0.960	5
400	decomposed	hybrid	0.690	0.867	0.585	0.590	0.893	162
400	probe	hybrid	0.564	0.596	0.397	0.417	0.711	50
400	raw_sql	hybrid	0.691	0.797	0.522	0.538	0.799	58
200	decomposed	dense	0.784	0.899	0.571	0.584	0.943	161
200	decomposed	bm25	0.710	0.837	0.637	0.627	0.963	7
200	decomposed	hybrid	0.640	0.878	0.604	0.569	0.948	163
300	decomposed	dense	0.764	0.676	0.511	0.548	0.934	162
300	decomposed	bm25	0.658	0.842	0.597	0.576	0.962	5
300	decomposed	hybrid	0.644	0.834	0.535	0.531	0.940	160
500	decomposed	dense	0.694	0.727	0.448	0.500	0.873	160
500	decomposed	bm25	0.728	0.839	0.589	0.604	0.968	4
500	decomposed	hybrid	0.647	0.854	0.566	0.564	0.874	166
600	decomposed	dense	0.743	0.645	0.458	0.532	0.932	162
600	decomposed	bm25	0.705	0.835	0.548	0.571	0.913	6
600	decomposed	hybrid	0.615	0.830	0.490	0.502	0.899	160

Breakdown by SQL feature (dataset label)

Feature	n	Baseline approval	RAG approval	Base checks	RAG checks
GROUP BY with HAVING	8	100.0% (8/8) [68-100]	100.0% (8/8) [68-100]	0.88	0.88
INNER JOIN of two tables	9	100.0% (9/9) [70-100]	100.0% (9/9) [70-100]	1.00	1.00
JOIN of three tables with ORDER BY and LIMIT	8	100.0% (8/8) [68-100]	100.0% (8/8) [68-100]	0.88	1.00
LEFT JOIN with IS NULL filter	3	100.0% (3/3) [44-100]	100.0% (3/3) [44-100]	1.00	1.00
simple SELECT with WHERE	5	100.0% (5/5) [57-100]	100.0% (5/5) [57-100]	1.00	1.00
subquery in WHERE	10	100.0% (10/10) [72-100]	100.0% (10/10) [72-100]	1.00	1.00
window function such as ROW_NUMBER or RANK	7	100.0% (7/7) [65-100]	85.7% (6/7) [49-97]	1.00	1.00

Items where the two conditions were graded differently

q001 · window function such as ROW_NUMBER or RANK

```
SELECT RANK() OVER (ORDER BY order_date) AS order_rank FROM orders;
```

Baseline: good

This query calculates a ranking of the orders based on their order date. The ranking is assigned to each order, with the earliest order getting a rank of 1, the next earliest order getting a rank of 2, and so on. The result is a list of orders with their corresponding ranking, where the order with the earliest order date has a rank of 1, the order with the next earliest order date has a rank of 2, and so on. The ranking is not based on any specific product or customer, but rather on the order date itself.

Judge: The explanation is correct, complete, clear, and grounded in the SQL and schema provided.

RAG: bad

This query ranks orders by their order date, assigning a unique rank to each distinct order date. The rank is calculated by the row_number() function, which assigns a number to each row within its partition (i.e., rows with the same order date), and then the rank() function assigns a rank to each row within its peer group (i.e., rows with the same order date and quantity). [S3]

Judge: The explanation incorrectly states that the row_number() function is used, when the SQL uses RANK(). The RANK() function assigns a unique rank to each row within its partition, not a number to each row within its peer group.